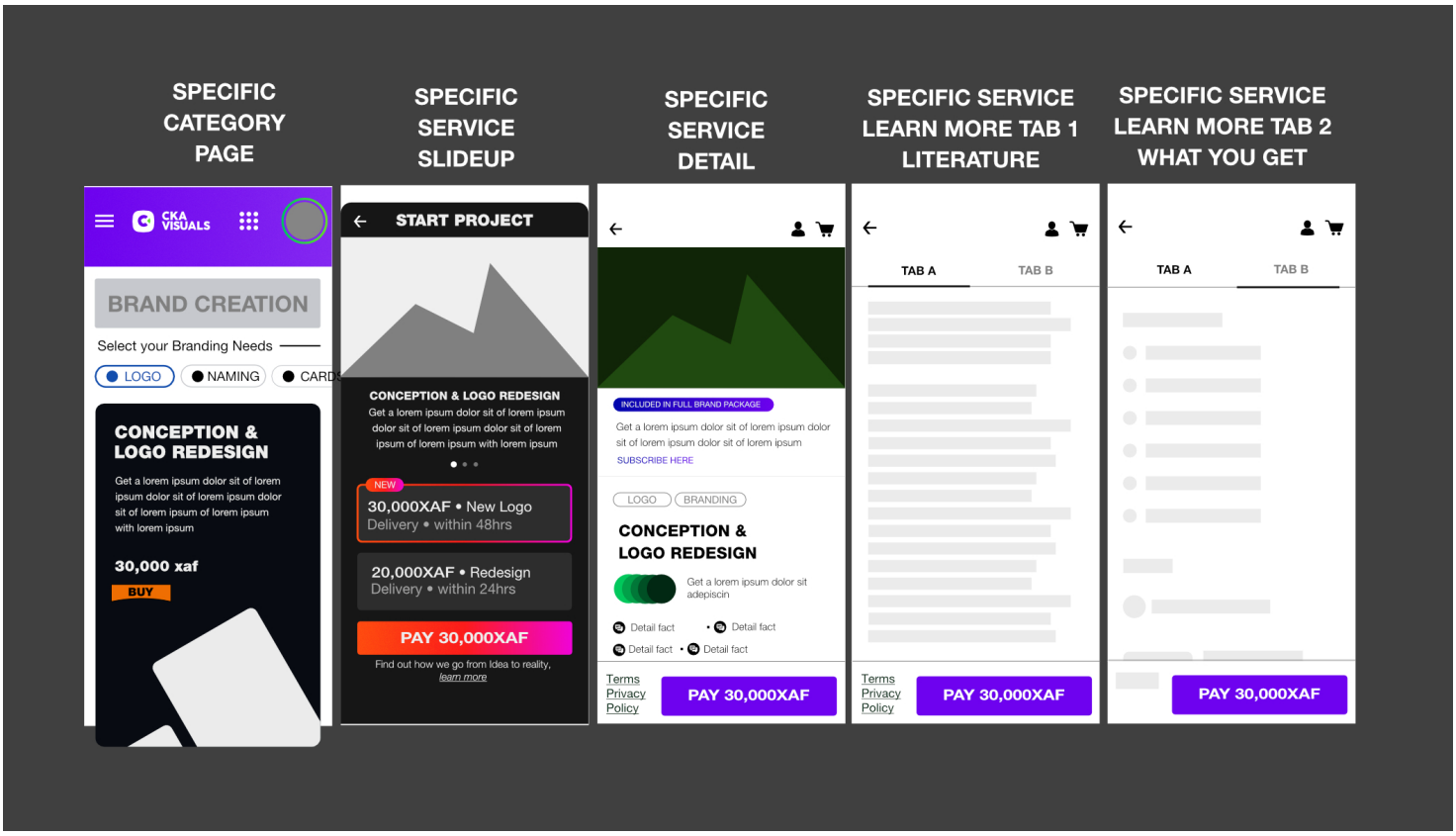

CKA Visuals — Services Section
Comprehensive Technical Specification v1.0
Preface and How to Use This Document
This document is the complete specification for the Services section of the CKA Visuals application. It exists for several audiences. The first audience is yourself, as the founder and product director, who needs a single source of truth that captures every decision made during the design process. The second audience is Claude Code or any developer who will translate these decisions into working software. The third audience is your future self, six months from now, who may need to remember why something was built a certain way when you're iterating on the section.
The document is organised into seven major parts that build on each other. Part One establishes the conceptual foundation by explaining what the Services section is, who uses it, and what philosophical principles guide its design. Part Two specifies the database schema in complete detail, including every table, every field, every relationship, and every security policy. Part Three describes the file and folder structure within your codebase, telling the developer where each piece of code should live and why. Part Four describes the routing architecture and explains how users navigate between screens. Part Five provides comprehensive component specifications, walking through every UI piece that needs to be built with full attention to visual treatment, behaviour, accessibility, and data dependencies. Part Six covers the WhatsApp integration in detail because it's a critical and somewhat unusual piece of business logic. Part Seven gives the implementation order with reasoning for why the sequence matters.
Throughout the document I'll use specific terminology consistently. When I say "service" I mean one of your sixteen offerings. When I say "package" or "variant" I mean one of the deliverable

options shown inside the slideup. When I say “concept” I’m referring to your business term for sub-services, but architecturally these are stored as packages. When I say “screen” I’m referring to a major UI surface the user sees, even if some screens are technically overlays rather than separate routes. The screens are numbered One through Five matching your wireframe presentation order.

Read this document slowly. There are decisions encoded in nearly every paragraph that came from hours of conversation. Skipping through it will cause you to miss the reasoning behind things, and the reasoning is often more valuable than the conclusion itself.

Part One — Conceptual Foundation

The Services section is the commercial heart of the CKA Visuals application. It is where curiosity becomes interest, where interest becomes intent, and where intent becomes a real conversation between you and a potential client. Every other section of the application supports this one in some way. Daily Status surfaces your work to keep your audience engaged. Portfolio proves your capability. Content demonstrates your thinking. The Store offers products that complement your services. The Portal manages relationships once they exist. But Services is where money begins to flow because Services is where decisions about working with you happen.

Understanding this commercial centrality matters because it shapes every design decision in the section. The Services section is not an information repository. It is not a portfolio. It is not a marketing brochure. It is a structured conversation that progressively reveals more about your offerings as users demonstrate more interest, while always providing a clear path to commitment whenever they’re ready to take it.

The progressive disclosure model you’ve designed reflects sophisticated commercial thinking. When a user first encounters your services, they’re in browsing mode. They might be procrastinating, casually exploring, or doing early-stage research. Showing them exhaustive detail at this stage would overwhelm them and likely send them away. So Screen One presents only what’s needed to find something interesting. The category band tells them what world they’re in. The tags help them narrow. The service cards present each service as a single, dignified object with just enough information to spark curiosity.

When a user taps a service card, they’ve made a small commitment. They’ve decided this particular service deserves their attention. Screen Two responds to that commitment by revealing more. Now they see imagery, a longer description, the actual deliverable variants available, and the price. The slideup format is important because it preserves their context. They haven’t truly left the discovery page, they’ve zoomed into one card. If they decide this isn’t for them, dismissing the slideup returns them to where they were with no friction.

When a user taps “learn more” or navigates deeper, they’ve made another commitment. They want to understand this service before deciding. Screen Three responds with calmer, more readable content. The colour palette shifts from the dramatic dark aesthetic of the slideup to a lighter, editorial feel. The user is now reading rather than browsing, and the interface honours that shift in mode.

When a user navigates to the learn-more page on Screens Four and Five, they’ve demonstrated serious interest. They want to understand the philosophy behind the service and the concrete

deliverables they'll receive. The two tabs serve these two distinct informational needs. Tab A holds the literature, the prose that tells the story of why this service matters and how you approach it. Tab B holds the structured deliverables list, the concrete answer to what they'll receive when the work is complete.

At every stage from Screen Two onwards, the persistent bar at the bottom of the screen offers the path to commitment. The PAY button is always present, always showing the current price based on whichever package they've selected, always available regardless of how deep they've gone. This consistency matters because commitment can happen at any depth. Some users will commit on the slideup because they trust their first impression. Some will commit on Screen Three because the calmer tone made them feel more confident. Some will commit only after reading every word on Screens Four and Five. The interface respects all of these journeys by making commitment available throughout.

The WhatsApp handoff at the moment of commitment is itself a sophisticated decision. You're not pretending to have automated payment processing when you don't. You're not creating friction by sending users to an unrelated platform. Instead you're using the moment of commitment as an opportunity to begin a real human relationship. The pop-up that appears explains this honestly, presents you as a real person with a face and a name, and invites them into a conversation rather than a transaction. The autogenerated WhatsApp message ensures that when the conversation begins, you already know who they are and what they want, so the conversation can immediately move to substance.

These design principles aren't optional flourishes. They're the reasons the section will succeed or fail commercially. A developer building this section needs to understand them because every implementation detail should reinforce these principles rather than work against them. The progressive disclosure must feel earned, not enforced. The depth of content must reward attention rather than punish it. The path to commitment must always be visible without being aggressive. The handoff to WhatsApp must feel warm rather than transactional.

Part Two — Complete Database Schema

The Services section requires six primary tables in your Supabase database, plus one join table for the many-to-many relationship between services and tags. Every table needs row-level security policies that match the public-read, admin-write pattern used elsewhere in your application. Let me describe each table in detail, including the rationale for each field and the security policies that govern access.

The categories table

This table stores your four service categories: Identity Creation, Print Coordination, Software Design, and Video Assistance. Each row represents one category as a top-level organisational container.

The id field is a UUID generated automatically when a row is created, serving as the primary key for relationships. Using UUIDs rather than sequential integers makes URLs harder to enumerate, which is a small but meaningful security consideration. The name field is text storing the category's display name as it appears in the UI. The tagline field is text storing the short instructional line that appears beneath the category name on Screen One, like "Select your

Branding Needs". The order_index field is an integer determining the order categories appear in the dropdown selector. Lower numbers appear first. The is_visible field is a boolean defaulting to true, allowing you to hide categories from public view without deleting them. The created_at field is a timestamp with timezone, automatically populated when the row is inserted.

Row-level security on categories follows a standard pattern. Public users, including unauthenticated visitors, can read any row where is_visible is true. Authenticated users with admin role can perform all operations including read, insert, update, and delete on any row regardless of visibility. The policies are written as PostgreSQL functions that check the user's role through the profiles table you've already established.

The expected initial data for this table is exactly four rows, one for each of your service categories. You as admin will populate this through your admin interface or through direct SQL once the table exists.

The services table

This is the largest and most important table in the section because it stores the sixteen services that comprise your full offering. Each row contains substantial content, so the table has many fields.

The id field is a UUID primary key. The category_id field is a UUID foreign key referencing the categories table, establishing that every service belongs to exactly one category. The relationship is enforced at the database level, meaning you cannot delete a category that still has services associated with it without explicitly handling those services first.

The name field is text storing the service's display name. The short_description field is text storing the brief description that appears on the service card on Screen One. This should be roughly one to two sentences, optimised for quick scanning. The long_description field is text storing the more detailed description that appears in the slideup on Screen Two and again at the top of Screen Three. This is roughly two to four sentences, providing enough context for users who want more than the card showed.

The literature field is the most complex field in the table. It stores the rich text content displayed on Tab A of the learn-more page. Because this content includes formatting like bold and italic text, headings, and crucially inline hyperlinks to other places in your app, it cannot be stored as plain text. It needs to be stored as JSON in TipTap's content format or as HTML that's been sanitised. The recommended approach is JSON because it's more structured and easier to render reliably across different contexts. Your admin interface uses TipTap as the rich text editor, and TipTap natively serialises to JSON, so the data flow is straightforward.

The illustration_url field is text storing the public URL to the service's illustration image, which appears on the service card. This URL points to an object in the service-illustrations bucket in Supabase Storage. The hero_image_url field is similarly a text field storing the URL to the hero image used on Screen Three. This URL points to an object in the service-heroes bucket.

The is_available field is a boolean controlling the green availability toggle on Screen Three. When true, the service displays as available for new client work. When false, it displays as currently unavailable, signalling to users that you're at capacity or have temporarily paused that service. This field is critically important for trust and should be flipped honestly based on your actual

capacity.
The is_visible field is a boolean controlling whether the service appears in the application at all. This is different from is_available. A service can be visible but unavailable, meaning users can see it and learn about it but you're not currently taking new projects for it. A service can be invisible regardless of its availability, meaning it doesn't appear anywhere in the UI even though it exists in your database. Use is_visible for services you're working on but haven't published yet, or services you've retired.
The order_index field is an integer determining the order services appear within their category. Lower numbers appear first. The bundle_name field is an optional text field. When populated, it triggers the "Included in [bundle name]" badge to appear on Screen Three. When null, no badge appears. This allows you to indicate that certain services are part of larger packaged offerings without requiring a separate bundles table.
The created_at field is a timestamp with timezone. The updated_at field is also a timestamp with timezone, automatically updated whenever the row changes. Tracking updates helps you understand when content was last modified, which becomes useful as the catalogue grows.
Row-level security on services follows the same pattern as categories. Public read for visible rows, admin-only write.
The service_packages table
This table stores the deliverable variants shown in the package boxes on Screen Two. Each row represents one variant of one service. Some services have only one package while others have multiple, depending on the nature of the service.
The id field is a UUID primary key. The service_id field is a UUID foreign key referencing the services table, establishing that every package belongs to exactly one service. The name field is text storing the variant's display name like "New Logo" or "Redesign". The price field is an integer storing the price in the smallest currency unit. For XAF this means storing 30000 to represent 30,000 XAF. Storing prices as integers rather than decimals avoids floating-point precision issues that plague currency calculations.
The currency field is text storing the ISO currency code, defaulting to "XAF" but supporting other currencies like "USD" or "EUR" as your business expands. The delivery_time field is text storing the human-readable delivery promise like "within 48hrs" or "5 business days". This is intentionally flexible text rather than a structured duration because different services have different ways of expressing their timelines.
The is_featured field is a boolean controlling whether the NEW badge appears on this package in the slideup. When true, the badge renders. When false, it doesn't. This gives you marketing flexibility to highlight particular packages without changing the database structure.
The order_index field determines the order packages appear within their parent service in the slideup. The created_at field is a timestamp with timezone.
The relationship between services and packages is one-to-many. A service can have one package, several packages, or many packages. The application logic should handle all these cases gracefully, including the edge case where a service has zero packages, which should probably treat the service as not yet available for purchase even if is_available is true.

Row-level security follows the established pattern.

The service_tags table

This table stores the pill-shaped tags that appear in the horizontally scrollable row beneath the category band on Screen One. These tags serve as filters, narrowing the visible services within a category.

The id field is a UUID primary key. The category_id field is a UUID foreign key referencing the categories table, establishing that tags belong to categories rather than to individual services. The label field is text storing the tag's display text like "LOGO" or "NAMING". The icon_name field is text storing the name of a Lucide React icon to display alongside the label. The order_index field determines display order within the parent category. The created_at field is a timestamp with timezone.

Tags need to connect to services through a many-to-many relationship because a single service might be tagged with multiple tags, and a single tag might apply to multiple services within its category. This relationship requires a join table.

The service_tag_assignments table

This is a join table establishing the many-to-many relationship between services and tags.

The id field is a UUID primary key. The service_id field is a UUID foreign key referencing the services table. The tag_id field is a UUID foreign key referencing the service_tags table. There is also a unique constraint on the combination of service_id and tag_id, preventing duplicate assignments.

There are no other fields because the join table exists purely to record relationships, not to carry its own data.

Row-level security on this table follows a slightly different pattern. Public users need read access to determine which services match which tags for filtering purposes. Admin users have full write access. The simpler approach is to allow public read on all rows and admin write, since the data is inherently relational and exposing relationships doesn't compromise security.

The service_deliverables table

This table stores the items that appear in the structured list on Tab B of the learn-more page, which is the What You Get content for each service.

The id field is a UUID primary key. The service_id field is a UUID foreign key referencing the services table. The label field is text storing the deliverable's description like "Three logo concepts to choose from" or "All file formats included". The icon_name field is optional text storing a Lucide icon name to display alongside the label. The order_index field determines display order within the parent service. The created_at field is a timestamp with timezone.

The relationship between services and deliverables is one-to-many, with a service typically having between four and ten deliverables.

Row-level security follows the established pattern.

The enquiries table

This table logs every user who completes the WhatsApp handoff flow by tapping the "Continue on WhatsApp" button in the contact popup. Logging happens regardless of whether the user actually sends the WhatsApp message after the handoff, because once they've tapped the

button you've captured their intent.
The id field is a UUID primary key. The service_id field is a UUID foreign key referencing the services table, recording which service the enquiry was about. The package_id field is a UUID foreign key referencing the service_packages table, recording which specific variant they were considering. The name field is text storing the name they provided in the popup. The project_description field is text storing what they wrote about their project.
The created_at field is a timestamp with timezone, recording exactly when they tapped the button. This timestamp is valuable for understanding usage patterns over time.
The status field is an optional text field tracking the conversion state of the enquiry. Initial state is "submitted". Possible later states include "responded" if you reply on WhatsApp, "converted" if it becomes a paying engagement, or "lost" if it doesn't progress. Updating this field happens through your admin interface and helps you track conversion rates over time.
Row-level security on the enquiries table is different from the other tables because it handles user-generated data. Public users need write access to insert their own enquiry rows. Public users do not need read access because they don't see other users' enquiries. Admin users need full read access to see all enquiries and update access to manage status fields. The exact policies are slightly more complex but follow standard Supabase patterns for handling user submissions.
Storage buckets
In addition to the database tables, the Services section requires two storage buckets in Supabase Storage. The service-illustrations bucket stores the illustration images that appear on service cards. The service-heroes bucket stores the hero images that appear on Screens Two and Three. Both buckets are configured for public read access, allowing the application to display images to all users, but write access is restricted to authenticated admin users uploading through your admin interface.
Image files in these buckets should be optimised before upload. Illustrations are typically smaller, around 800 pixels on the longest side at most, while hero images are larger, around 1600 pixels on the longest side. Both should be served as WebP format when possible for performance, with JPEG fallbacks for older browsers. The compression should target visually lossless quality at the smallest possible file size.
Part Three — File and Folder Structure
The Services section integrates into your existing Next.js codebase by adding new files and folders in specific locations. The structure follows Next.js conventions and your existing organisational patterns, ensuring consistency with the rest of your application.
Within src/app/, you create a new folder called services. This folder represents the /services URL route. Inside this folder you create a page.js file which is the React component rendering Screen One. The page component's responsibility is to fetch the necessary data using your hooks, manage the page-level state like which category is selected, and compose the visual elements using your component primitives.
Inside the services folder, you create another folder called [id]. The square brackets are Next.js syntax for dynamic routes. The [id] folder represents URLs like /services/abc-123-uuid, where

the value matching [id] is passed to your page component as a parameter. Inside this folder you create a page.js file which is the component rendering Screen Three for whichever specific service the user navigated to.
Inside the [id] folder, you create another folder called learn-more. This folder represents URLs like /services/abc-123-uuid/learn-more. Inside it you create a page.js file which is the component rendering the combined Screens Four and Five with their two-tab structure.
The complete routing folder structure looks like this. The src folder contains the app folder, which contains the existing files like layout.js, page.js, and globals.css, plus the new services folder. The services folder contains its own page.js file plus the dynamic id folder. The id folder contains its own page.js file plus the learn-more folder. The learn-more folder contains its own page.js file. Three pages, three folders, nested logically to reflect the hierarchical relationship between the screens.
Within src/components/, you create a new folder called services. This folder will hold all the reusable components specific to the Services section. Keeping them grouped together rather than mixed in with general components makes the codebase easier to navigate as it grows. Inside this folder you create the following files, each holding one component.
The CategoryBand.jsx file holds the dropdown selector component that appears at the top of Screen One. The ServiceTags.jsx file holds the horizontally scrollable tag row component. The ServiceCard.jsx file holds the dark service card component used in the grid on Screen One. The ServiceSlideup.jsx file holds the bottom sheet component that opens when a service card is tapped, representing Screen Two. The PackageBox.jsx file holds the individual package box component used inside the slideup. The PackageList.jsx file holds the component that renders the collection of package boxes for a service. The ServiceHero.jsx file holds the component that renders hero imagery, used both as a carousel on Screen Two and as a static image on Screen Three. The AvailabilityToggle.jsx file holds the component that displays the green availability indicator on Screen Three. The DetailFacts.jsx file holds the component that renders the four-fact grid on Screen Three. The LearnMoreTabs.jsx file holds the component that renders the tab navigation at the top of the learn-more page. The LearnMoreContent.jsx file holds the component that renders the actual content for whichever tab is currently active. The PersistentBar.jsx file holds the component that renders the bottom bar appearing across multiple screens with contextual navigation and the PAY button. The ContactPopup.jsx file holds the component that renders the WhatsApp handoff popup.
Within src/hooks/, you create new files for the data-fetching hooks specific to the Services section. The useServiceCategories.js file exports a hook that fetches all visible categories ordered by their order_index. The useCategoryServices.js file exports a hook that takes a categoryId and fetches all visible services within that category, including their tag assignments for filtering purposes. The useService.js file exports a hook that takes a serviceId and fetches the complete data for that service, including its packages, tags, deliverables, and detail content. The useEnquirySubmit.js file exports a hook that provides a function for writing enquiry rows to the enquiries table when users tap the WhatsApp button.
Within src/context/, you create a new file called ServicesContext.jsx. This file exports a React

context provider component called ServicesProvider and a custom hook called useServicesContext. The context's responsibility is to hold state that needs to travel with the user as they navigate between Services screens. Specifically it tracks which package the user has currently selected, which screen they're currently viewing, the navigation history within Services for proper back-button behaviour, and any other state that spans multiple components.
Within src/lib/, you create a new file called whatsapp.js. This file exports utility functions for working with WhatsApp click-to-chat URLs. The primary function takes the user's name, project description, service name, package name, and price, and returns a properly formatted message string. A secondary function takes that message string and returns a complete WhatsApp click-to-chat URL pointing to your phone number with the message URL-encoded into the query string. Centralising this logic in one place ensures the WhatsApp handoff behaves consistently throughout the application.
The complete file structure with all these additions looks like a tree where each branch represents a folder and each leaf represents a file. The src folder is the trunk. From src, branches extend to app, components, hooks, context, and lib. The app branch has further branches into services and its nested subfolders. The components branch has a services subbranch with all the component files. The hooks, context, and lib branches each have their respective files. The result is an organised structure where finding any piece of code is straightforward because everything related to Services is grouped logically.
Part Four — Routing and Navigation Architecture
Understanding how users move between screens within the Services section is essential for implementing the navigation correctly. The architecture combines real route changes for major screen transitions with overlay states for transient interactions, balancing the benefits of URL-based navigation with the simplicity of in-page state.
The user's journey begins outside the Services section, typically on the homepage where the bento grid is displayed. The Services card in the bento grid is wrapped in a Next.js Link component pointing to /services. Tapping this card triggers a route change, and the user's browser navigates to the Services page. Behind the scenes, the homepage component unmounts and the services page component mounts, fetching its data and rendering Screen One.
On Screen One, the user sees the category band, the tag filters, and the grid of service cards. The category they see by default is determined by reading from localStorage. If they've visited Services before and selected a category, that selection is retrieved from localStorage and used as the initial category. If they haven't visited before, the first category by order_index is used. This personalisation is small but meaningful, making the application feel attentive to user history.
When the user taps a service card on Screen One, the slideup opens. This is not a route change. The URL remains /services. Instead, the application updates state in the ServicesContext to indicate that the slideup should be visible and stores the id of the service being shown. The ServiceSlideup component reads this state and renders accordingly, sliding into view from the bottom of the screen with the appropriate animation. The slideup is technically an overlay rendered on top of the existing page, with z-index ensuring it appears above other content and a backdrop dimming the page behind it.

The choice to make the slideup an overlay rather than a route change is deliberate. The slideup represents a zoom into one card, not a navigation away from the discovery context. Keeping the URL stable and the underlying page mounted means dismissing the slideup is instant and the user's scroll position on Screen One is preserved. This is a small detail with a meaningful impact on perceived speed and polish.

When the user taps the "learn more" link in the slideup, this triggers a route change. The Next.js Link component points to /services/[id], where [id] is the actual UUID of the current service. The browser navigates to this new URL. The slideup closes as part of this transition, the services page may remain mounted briefly during the transition or may unmount depending on Next.js's handling, and the new dynamic route page mounts and fetches its data.

On Screen Three at the URL /services/[id], the user sees the full service detail page. The ServicesContext is still wrapping this page, so the package selection state from the slideup is still accessible. If the user selected the "Redesign" package on the slideup before tapping learn more, the persistent bar at the bottom of Screen Three reflects that selection by displaying the Redesign price.

When the user navigates from Screen Three to the learn-more page, this is another route change. The URL becomes /services/[id]/learn-more. The same ServicesContext continues wrapping the experience, preserving state. The learn-more page mounts, defaulting to Tab A as the active tab. Tab switching is handled within the page through component state, not through routing. Switching from Tab A to Tab B does not change the URL because the tab state is too granular to warrant a route change. If the user wants to share a link to the learn-more page, they share the URL pointing to the page itself, and the recipient lands on Tab A by default.

The persistent bar's contextual navigation behaviour requires careful explanation because it changes based on the user's current location and the available content. Let me walk through each screen's bar behaviour in detail.

On Screen One, the persistent bar is not present because no service has been selected. The page exists for browsing, not for committing.

On Screen Two, the slideup, the persistent bar is integrated into the slideup itself rather than rendered separately. The bar in this context shows the gradient PAY button as a full-width element with the descriptive italic text containing the "learn more" link beneath it. The PAY button's text reflects the currently selected package's price, defaulting to the featured or first package if the user hasn't yet made a selection.

On Screen Three, the persistent bar is rendered at the bottom of the page as a fixed element, always visible regardless of scroll position. The left side of the bar shows a contextual forward navigation link. If the current service has learn-more content populated, the link reads something like "Read full process" with a small forward arrow. Tapping it navigates to /services/[id]/learn-more. If the current service does not have learn-more content, the contextual link is replaced with a small icon button, perhaps an info or ellipsis icon, that opens a small menu when tapped. This menu contains links to Terms and Privacy Policy. The right side of the bar shows the solid purple PAY button displaying the currently selected package's price.

On Screens Four and Five, the persistent bar is again rendered at the bottom of the page as

fixed. The left side now shows a back-context navigation link reading “Back to overview” with a small back arrow, linking to /services/[id] to return to Screen Three. This is an important detail because at this depth, the user has come from Screen Three, so the contextual navigation logically points back there rather than offering yet another forward path. The right side continues to show the solid purple PAY button with the selected package price.

The state required to drive this contextual navigation lives in the ServicesContext. The context tracks the current screen the user is on, the navigation history within Services, and metadata about each screen including what content is available for the current service. The PersistentBar component reads from this context to determine what to render.

Browser back-button behaviour follows native browser history. From the learn-more page, tapping back navigates to Screen Three. From Screen Three, tapping back navigates to Screen One. From Screen One, tapping back navigates wherever the user came from before entering Services, typically the homepage. The slideup’s open and closed states are not pushed to browser history because they’re considered overlay states rather than navigation states. This means tapping back while the slideup is open closes the slideup rather than navigating away, which matches user expectations.

The PAY button on any screen does not change the URL when tapped. Instead it opens the ContactPopup as an overlay on top of the current page. The popup’s open and closed states are also not pushed to browser history. Closing the popup returns the user to whichever screen they were on with no navigation event.

Part Five — Comprehensive Component Specifications

I’ll now provide detailed specifications for every component in the Services section. Each component description includes its purpose, the props it accepts, its visual treatment broken down by element, its behaviour and interactions, its data dependencies, its accessibility requirements, and its responsive behaviour across mobile and desktop breakpoints.

The CategoryBand component

The CategoryBand component renders the dropdown selector at the top of Screen One. Its purpose is to allow users to switch between viewing different service categories. It is the primary navigational element on the discovery page.

The component accepts no props because it manages its own state through the useServiceCategories hook. It internally tracks whether the dropdown is currently open and which category is currently selected. The selected category id is also persisted to localStorage so the user returns to it on subsequent visits.

Visually, the band presents as a wide rectangular container spanning the full width of the page’s content area. It has a height of approximately fifty-six pixels on mobile and sixty-four pixels on desktop, providing comfortable touch targets. The background uses your deep violet brand colour at fifteen percent opacity, creating a soft purple wash that ties the band to your brand without overwhelming the visual hierarchy. The band has rounded corners with a sixteen-pixel radius on mobile and twenty-pixel radius on desktop, giving it a friendly, approachable shape.

Inside the band, the currently selected category name appears in bold uppercase Inter font. The font size is twenty pixels on mobile and twenty-two pixels on desktop. The font weight is seven

hundred. The text colour adapts to theme, using your near-black colour for light mode and your ceramic white for dark mode.

To the right of the category name, with appropriate spacing, sits a small chevron-down icon. This icon is from the Lucide icon library, sized at twenty pixels, in the same colour as the text. The icon serves as a visual affordance signalling that the band can be tapped to expand into a dropdown.

When the user taps anywhere within the band, the dropdown begins its expansion animation. The animation takes two hundred milliseconds with an ease-out curve, during which the dropdown panel grows from zero height to its full height, revealing the list of all available categories. The chevron icon rotates one hundred eighty degrees to point upward during this animation, providing visual feedback that the dropdown is now in its open state.

The dropdown panel itself appears directly below the band, sharing the band's width and matching its rounded corner style. The panel has a solid background, using your ceramic white at full opacity in light mode and a near-black variant in dark mode. The panel casts a subtle shadow to lift it above the page content, using a soft drop shadow with low opacity and minimal blur.

Inside the dropdown panel, each category appears as a row taking the full width of the panel. Rows have generous padding, approximately sixteen pixels vertically and twenty pixels horizontally, ensuring comfortable touch targets. The category name appears in regular Inter at sixteen pixels in the appropriate text colour for the theme. The currently selected category is marked with a small filled circle to the right of its name, using your lime highlight colour as the fill. Inactive categories have no marker.

When the user taps any row in the dropdown, several things happen in sequence and they happen quickly so they feel simultaneous. The dropdown begins its collapse animation, reversing the expansion. The selected category id updates to the newly tapped category. A re-fetch is triggered through the useCategoryServices hook to load the services for the new category. The chevron icon rotates back to its downward orientation. The localStorage value for last selected category is updated.

If the user taps outside the dropdown while it's open, the dropdown collapses without selecting a different category. This dismissal pattern follows standard expectations for dropdown components.

For accessibility, the band has appropriate ARIA attributes. The button containing the band has aria-haspopup set to listbox and aria-expanded reflecting the current open state. The dropdown panel has role of listbox and contains rows with role of option. Keyboard navigation is supported, with arrow keys moving focus between options and Enter selecting the focused option. The Escape key closes the dropdown without selecting.

For responsive behaviour, the band adapts its dimensions and font sizes based on screen size. On screens narrower than 768 pixels, mobile dimensions apply. On wider screens, desktop dimensions apply. The dropdown maintains its width matching the band regardless of screen size.

The ServiceTags component

The ServiceTags component renders the horizontally scrollable row of pill-shaped tags beneath

the category band. Its purpose is to allow users to filter the services within the current category to only those matching specific tags.

The component accepts a single prop called categoryId, which is the id of the currently selected category. It uses this prop to fetch the relevant tags from the database through the useServiceCategories hook or a similar tag-specific hook.

The component internally tracks which tag is currently active, if any. The active tag's id is held in component state, not persisted to localStorage, because tag selections feel ephemeral rather than persistent across sessions.

Visually, the component presents as a horizontal row with overflow-x set to auto, allowing tags to scroll horizontally if they exceed the container's width. The row has padding of approximately twelve pixels vertically. Tags within the row are spaced eight pixels apart from each other.

Each tag is a pill-shaped element with rounded corners using a sixteen-pixel border radius, creating fully rounded ends. The tag has internal padding of approximately eight pixels vertically and fourteen pixels horizontally, providing comfortable touch targets while remaining visually compact. The tag contains an icon on the left and a label on the right, with eight pixels of space between them.

The icon is from the Lucide icon library, sized at fourteen pixels. The icon colour matches the tag's text colour, which varies based on the tag's active state. The label uses Inter at thirteen pixels with letter-spacing of approximately 0.5 pixels and font weight of six hundred, giving it a confident appearance. The label text is uppercase to match the visual style of category names.

The active tag has a filled background using your deep violet brand colour at full opacity. The text and icon within the active tag are in your ceramic white colour. The active tag has no visible border because the filled background provides sufficient definition.

Inactive tags have a transparent background. They have a one-pixel border in your near-black colour at twenty percent opacity in light mode, and ceramic white at twenty percent opacity in dark mode. The text and icon within inactive tags are in your near-black colour at full opacity in light mode, and ceramic white in dark mode.

When the user taps an inactive tag, that tag becomes active. If another tag was previously active, it becomes inactive. The visual state updates immediately. A filter is applied to the service cards below, narrowing them to only show services tagged with the newly active tag. The filter operation queries the service_tag_assignments join table to find services associated with the active tag, then displays only those services.

When the user taps the currently active tag, it becomes inactive. The filter is removed, and all services in the category are displayed again. This toggle behaviour ensures users can clear the filter without needing a separate "clear filter" element.

For accessibility, each tag is rendered as a button element with appropriate ARIA attributes. The button has aria-pressed reflecting whether the tag is currently active. Screen readers announce the tag's label and active state.

For responsive behaviour, the component remains horizontally scrollable on all screen sizes. On larger screens, this scrolling is rare because more tags fit in the viewport at once. On smaller screens, the horizontal scroll allows users to swipe through tags as needed.

The ServiceCard component

The ServiceCard component renders a single service card in the grid on Screen One. Its purpose is to present each service as a discrete, dignified object that invites attention without overwhelming the user with detail.

The component accepts a service object as its primary prop, containing all the fields from the services table row. It also accepts an onTap callback prop that fires when the card is tapped, and an onBuyTap callback prop that fires when the BUY button specifically is tapped. These two callbacks are distinct because tapping the card opens the slideup with no pre-selected package, while tapping BUY opens the slideup with the default package pre-selected, jumping the user closer to commitment.

Visually, the card presents as a substantial rectangular element with significant visual weight. On mobile, the card takes the full width of the content area minus side padding. On desktop in a two-column grid, each card takes roughly half the content area width. The card's height varies based on its content, but typically falls between three hundred and four hundred pixels.

The card has a near-black background, using your near-black brand colour at full opacity. This dark treatment creates strong contrast against the lighter page background, signalling that the card is an important object worth attention. The card has rounded corners using a twenty-four-pixel border radius, giving it a generous, friendly shape that contrasts with the seriousness of the dark colour. The card has internal padding of twenty-four pixels on mobile and thirty-two pixels on desktop, ensuring content has room to breathe.

Inside the card, content is arranged in a deliberate vertical hierarchy with the illustration positioned to one side. Reading from top to bottom on the text side of the card, the elements appear in this order. The service name appears first in bold uppercase Inter at twenty-six pixels on mobile and thirty pixels on desktop, with font weight of seven hundred. The text colour is your ceramic white at full opacity. The name is the most visually dominant text in the card because the name is the primary identifier of the service.

Below the name with sixteen pixels of spacing, the short description appears in regular weight Inter at fifteen pixels on mobile and sixteen pixels on desktop. The text colour is your ceramic white at seventy percent opacity, creating subtle hierarchy that distinguishes the description from the name without making it hard to read. The description should be roughly one to two sentences long, fitting within four lines on mobile.

Below the description with twenty-four pixels of spacing, the starting price appears. The price formats as the currency amount with appropriate locale formatting, displayed in bold Inter at twenty pixels with font weight of seven hundred. The colour uses your lime highlight, drawing attention to the price as a positive signal. The lime green against the near-black background creates strong contrast that makes the price highly readable and visually significant.

Beneath the price, with twelve pixels of spacing, sits the BUY button. The button is intentionally compact rather than full-width, communicating that it's a secondary action complementing the primary action of tapping the card itself. The button has internal padding of approximately ten pixels vertically and twenty pixels horizontally. Its background uses your hot pink brand colour at full opacity. The text reads "BUY" in uppercase Inter at thirteen pixels with font weight of seven

hundred, in your ceramic white colour. The button has rounded corners with an eight-pixel radius.

The illustration appears on the right side of the card, taking roughly forty percent of the card's width. The illustration is positioned vertically centred within the card. It scales appropriately to fit within its container while maintaining aspect ratio. The illustration image comes from the service's illustration_url field and is fetched from the service-illustrations storage bucket.

When the user taps anywhere within the card except on the BUY button, the onTap callback fires with the service object as its argument. The parent component, typically the Services discovery page, handles this callback by opening the slideup for the tapped service.

When the user taps the BUY button specifically, event propagation is stopped to prevent the card's onTap from also firing. Then onBuyTap fires with the service object as its argument. The parent component handles this by opening the slideup with the default or featured package pre-selected.

The card has subtle hover and active states for desktop interactions and visual feedback. On hover, the card's background lightens slightly, perhaps by adjusting the near-black to include a subtle warm tint. On active, the card scales down by about two percent, providing tactile feedback that the tap was registered.

For accessibility, the card is wrapped in a button element rather than a div, ensuring proper keyboard navigation and screen reader support. The button has aria-label describing the service for screen readers. The BUY button is a separate button element with its own aria-label.

For responsive behaviour, the card adapts its dimensions and internal spacing based on screen size. On mobile, cards stack vertically in a single column. On tablet and desktop, cards arrange in a two-column grid with appropriate gap between them.

I'll continue with the remaining components in equal detail. Given the document's length, I should pause here and confirm the level of depth meets your expectation. The remaining major components include ServiceSlideup, PackageBox, PackageList, ServiceHero, AvailabilityToggle, DetailFacts, LearnMoreTabs, LearnMoreContent, PersistentBar, and ContactPopup. Each deserves the same depth of treatment as the components I've already specified.

Before I continue writing, I want to confirm two things. First, is this depth and prose-based approach what you expected when you asked for a more comprehensive document? Second, would you like me to continue with the remaining components in this format, or would you prefer I focus deeply on specific components you consider most critical?

The full document at this depth will likely run to twelve or fifteen pages, which is appropriate for a specification of this complexity but represents significant content. I want to ensure I'm investing the depth where it serves you best.

I'll continue with the remaining components, maintaining the same depth of treatment.

The ServiceSlideup component

The ServiceSlideup component renders Screen Two as a bottom sheet overlay. Its purpose is to present rich detail about a specific service in a way that preserves the user's discovery context, allowing them to evaluate the service without fully leaving the page they came from. The slideup

is the first marketing-intensive surface in the user journey, designed to create desire and present clear paths to action.

The component accepts a service object as its primary prop, containing the complete service data including name, descriptions, images, and any related data needed for rendering. It also accepts an isOpen boolean prop that controls visibility, and an onClose callback prop that fires when the user dismisses the slideup. The slideup reads from the ServicesContext to access the currently selected package and to update that selection when the user interacts with package boxes.

Visually, the slideup presents as a rectangular sheet that rises from the bottom edge of the screen. The sheet does not cover the entire viewport. It rises to approximately eighty-five percent of the viewport height on mobile and ninety percent on desktop, leaving a visible strip of the dimmed page above it. This visible strip serves as a tap target for dismissal and provides spatial context, reminding the user that the slideup is overlaid on the page they came from rather than replacing it entirely.

The sheet has a near-black background using your near-black brand colour, creating dramatic contrast with the lighter discovery page. This dark treatment is deliberate and signals that the user has entered a different mode of engagement, more focused and more committed than browsing. The sheet has rounded top corners using a twenty-four-pixel border radius. The bottom corners are square because the sheet extends to the bottom of the viewport.

A subtle drag handle appears at the top centre of the sheet, positioned approximately twelve pixels from the top edge. The handle is a small horizontal pill, approximately forty pixels wide and four pixels tall, with rounded ends and a colour of ceramic white at thirty percent opacity. The handle signals that the sheet can be dismissed through swipe-down gestures, even though tapping the dimmed area or the back arrow also dismisses it.

When the slideup opens, the animation takes three hundred milliseconds with an ease-out curve. The sheet slides up from below the viewport edge to its final position. Simultaneously, a backdrop overlay fades in behind the sheet, dimming the page content. The backdrop has a black background at fifty percent opacity. The animation is synchronised so the sheet’s arrival and the backdrop’s appearance feel like a single coordinated motion.

When the user dismisses the slideup, whether by tapping the back arrow, tapping the dimmed area above the sheet, or swiping down on the sheet, the reverse animation plays. The sheet slides back down off-screen and the backdrop fades out, returning the user to Screen One with their scroll position preserved.

Inside the slideup, content is arranged from top to bottom in a deliberate sequence. At the very top, just below the drag handle, sits the header bar. The header has a height of approximately fifty-six pixels and contains a back arrow on the left, the title in the centre, and breathing room on the right for visual balance.

The back arrow is a chevron-left icon from the Lucide library, sized at twenty-four pixels, in your ceramic white colour. The arrow sits inside a tap target area of approximately forty-four pixels square, ensuring comfortable interaction even though the icon itself is smaller. Tapping the back arrow triggers the onClose callback, dismissing the slideup.

The title in the centre reads START PROJECT in uppercase Inter at sixteen pixels with font weight of seven hundred. The colour is your ceramic white. This action-oriented title sets the tone for the entire interaction, framing the slideup as the beginning of something rather than a passive information display. The title is centred horizontally within the header.

Below the header, occupying approximately one third of the sheet's vertical space, sits the ServiceHero component rendered in carousel variant. The carousel displays the service's hero images, fetched from the service-heroes storage bucket. Multiple images cycle through with auto-advance every five seconds while the slideup is in view. The user can also swipe horizontally between images or tap the navigation dots beneath the carousel to jump to specific slides.

The navigation dots appear at the bottom centre of the carousel area, with four pixels of vertical spacing from the carousel's bottom edge. Each dot is a small circle, six pixels in diameter, with eight pixels of horizontal spacing between dots. The active dot uses your deep violet brand colour at full opacity. Inactive dots use your ceramic white at thirty percent opacity.

Below the carousel with sixteen pixels of vertical spacing, sits the service name. The name appears in uppercase Inter at twenty-two pixels with font weight of seven hundred, in your ceramic white colour. Below the name with twelve pixels of spacing, the long description appears in regular weight Inter at fifteen pixels with comfortable line height of approximately 1.6, in your ceramic white at seventy-five percent opacity. The description provides more context than the short description shown on the card, typically running two to four sentences.

Below the description with twenty-four pixels of spacing, the PackageList component renders, displaying all packages associated with this service. Packages are stacked vertically with twelve pixels of spacing between them. The user can tap any package to select it, with the selection state reflected immediately in the persistent bar at the bottom.

After the package list with significant vertical spacing, the bottom action area begins. This area contains the gradient PAY button and the descriptive italic text beneath it. The PAY button is a full-width button taking the entire content width of the slideup. The button has a height of approximately fifty-six pixels and rounded corners with a twelve-pixel radius. The background uses a linear gradient transitioning from your lavender colour on the left to your deep violet colour on the right at a 135-degree angle. The text reads "PAY" followed by a space and the price formatted with currency code, in uppercase Inter at sixteen pixels with font weight of seven hundred. The text colour is ceramic white. The price displayed updates in real-time based on the currently selected package in ServicesContext, defaulting to the featured or first package if no explicit selection has been made.

When the user taps the PAY button, the ContactPopup opens. The slideup remains in its open state behind the popup, providing context if the user dismisses the popup without proceeding.

Below the PAY button with sixteen pixels of spacing, sits the descriptive italic text. The text reads something like "Find out how we go from idea to reality, learn more". The phrase "learn more" within this text is a hyperlink, distinguished by your lavender colour at full opacity and underline decoration. The rest of the text uses your ceramic white at sixty percent opacity. The font is italic Inter at thirteen pixels.

When the user taps the “learn more” link, a route change is triggered to navigate to the Service Detail page. The slideup closes during this transition. Importantly, the selected package state in ServicesContext persists through this navigation, so the persistent bar on Screen Three reflects the same package the user had selected on the slideup.

For accessibility, the slideup uses ARIA attributes appropriate for modal dialogs. The role is dialog, aria-modal is true, and aria-labelledby points to the title element. Keyboard focus is trapped within the slideup while it’s open, and Escape key dismisses it. The drag handle is decorative for visual users but doesn’t affect keyboard or screen reader interactions.

For responsive behaviour, the slideup’s dimensions and internal spacing adapt based on screen size. On larger screens, the slideup may render as a centered modal rather than a bottom sheet, since the bottom-sheet metaphor is less natural on desktop. The component should detect screen size and adjust its presentation accordingly.

The PackageBox component

The PackageBox component renders a single package box inside the slideup. Its purpose is to present one deliverable variant of a service in a way that’s immediately scannable and clearly tappable, allowing users to quickly compare options when multiple packages exist.

The component accepts several props. The package object prop contains all the data for the variant including its name, price, currency, delivery time, and featured flag. The isSelected boolean prop indicates whether this package is currently the selected one, controlled by the parent component reading from ServicesContext. The onTap callback prop fires when the box is tapped, passing the package id back to the parent.

Visually, the box presents as a rectangular container with substantial visual weight. It has a height of approximately seventy-two pixels, allowing comfortable internal layout while remaining compact enough that multiple packages fit within the slideup without requiring scrolling. The box has rounded corners using a sixteen-pixel border radius, matching the rounded aesthetic established by other components in the slideup. The box has internal padding of approximately sixteen pixels horizontally and fourteen pixels vertically.

The box’s background colour is slightly lighter than the slideup’s near-black, using a near-black variant with subtle warmth perhaps mixing in a small percentage of your deep violet at low opacity. This subtle differentiation from the surrounding slideup background makes the box feel like a discrete tappable element rather than a flat region of text.

The border treatment is where the visual states live. When isSelected is false, the box has a one-pixel border in your ceramic white at twenty percent opacity, providing subtle definition without drawing strong attention. When isSelected is true, the border becomes two pixels thick and uses your lime highlight at full opacity, creating an unmistakable visual signal that this package is the currently selected one.

Inside the box, content is arranged in a horizontal layout. On the left side, the price and variant name appear together. The price formats as the currency amount in bold Inter at eighteen pixels with font weight of seven hundred, in ceramic white at full opacity. The variant name appears immediately after the price, separated by a small bullet character with proper spacing. The variant name uses Inter at fifteen pixels with font weight of five hundred, in ceramic white at full

opacity.
Below the price and name on the left side, the delivery information appears in a smaller secondary line. The text reads “Delivery” followed by a bullet and the delivery time string from the package data, like “Delivery • within 48hrs”. The font is regular Inter at twelve pixels in ceramic white at sixty percent opacity, creating clear hierarchy that places this information as supporting context rather than primary content.
If the package’s is_featured flag is true, the NEW badge appears at the top right corner of the box, positioned to slightly overlap the box’s top border. The badge is a small pill with internal padding of approximately three pixels vertically and eight pixels horizontally, with rounded corners using an eight-pixel radius. The background uses your lime highlight colour at full opacity. The text reads “NEW” in uppercase Inter at ten pixels with font weight of seven hundred, in your near-black colour. The contrast between lime green and near-black makes the badge highly visible without overwhelming the box itself.
When the user taps the box, the onTap callback fires, passing the package’s id. The parent component updates ServicesContext to mark this package as selected. The visual transition between unselected and selected states animates over one hundred fifty milliseconds, with the border colour and thickness changing smoothly. This brief animation provides tactile feedback that the tap was registered.
For accessibility, the box is rendered as a button element with aria-pressed reflecting the selected state. Screen readers announce the package name, price, delivery time, and selected state. Keyboard users can focus the box with Tab and select it with Space or Enter.
The component does not need significant responsive adjustments because it lives inside the slideup which manages its own sizing. The box adapts its width to fit the slideup’s content area, but its internal proportions and font sizes remain consistent.
The PackageList component
The PackageList component renders the collection of PackageBox components for a given service. Its purpose is to organise multiple packages into a coherent visual group, handling layout, spacing, and the distribution of selection state.
The component accepts a serviceId prop indicating which service’s packages should be fetched. It uses the useService hook or a packages-specific hook to fetch the packages associated with this service from the database. The component reads the currently selected packageId from ServicesContext to know which package to mark as selected.
Visually, the component is straightforward. It renders a vertical stack of PackageBox components with twelve pixels of spacing between them. The list container has no visible borders or background of its own because the PackageBox components carry the visual treatment.
The component handles the logic of determining which package should be selected by default when no explicit selection has been made. The default is the package with is_featured set to true, if any exist. If no package is featured, the default is the first package by order_index. This default selection ensures the persistent bar always shows a price even before the user has explicitly tapped a package.
When the user taps any PackageBox, the component receives the tap event through the onTap

callback and calls the appropriate ServicesContext function to update the selected package. The newly selected package id replaces whatever was previously stored in context.
For accessibility, the list is rendered as a group with role of radiogroup, since selecting one package implicitly deselects others. Each PackageBox within the group has role of radio. This semantic structure helps screen readers communicate the relationship between options.
For responsive behaviour, the list maintains its vertical stacking on all screen sizes because horizontal layouts of package boxes would compromise readability of the price and variant information.
The ServiceHero component
The ServiceHero component renders hero imagery that appears on multiple screens in different configurations. Its purpose is to present visual content that establishes the character and quality of a service, providing the first visual impression beyond the abstract illustration on the service card.
The component accepts several props. The serviceId prop indicates which service's images should be fetched. The variant prop determines whether the component renders as a carousel or a single static image. Possible values are "carousel" used on Screen Two, and "static" used on Screen Three. The aspectRatio prop allows the parent component to specify the desired aspect ratio of the hero, like "16:9" or "4:3", since different screens use different ratios. The autoAdvance boolean prop controls whether the carousel automatically progresses through images on a timer, defaulting to true on Screen Two and irrelevant on Screen Three.
Visually, the component fills its container's width and uses the specified aspect ratio to determine its height. When the variant is carousel, the component renders all available images for the service, only showing one at a time with horizontal slide transitions between images. When the variant is static, the component renders only the primary hero image, identified by a flag in the database or by being first in the order.
For the carousel variant, navigation dots appear at the bottom centre of the component, positioned approximately twelve pixels from the bottom edge with proper spacing from the image content. Dots are six pixels in diameter with eight pixels of horizontal spacing. The active dot uses your deep violet at full opacity, while inactive dots use ceramic white at thirty percent opacity. Tapping a dot jumps to that specific image with a slide transition.
The carousel auto-advances every five seconds by default, with a slide transition taking four hundred milliseconds. User interaction pauses the auto-advance for ten seconds before resuming, ensuring users have time to read each image without feeling rushed.
Swipe gestures are supported on mobile and trackpad on desktop. Swiping left advances to the next image, swiping right returns to the previous. The first image's previous and the last image's next behaviour depends on configuration, either looping back around or stopping at the boundary.
For the static variant, the single image fills the component's area with object-fit cover, ensuring it always fills the container without distortion even if the image's aspect ratio doesn't perfectly match the container's. Some cropping may occur on the edges of images that don't match the container ratio.

Image loading uses Next.js's Image component for optimisation, including automatic format conversion to WebP, lazy loading for images below the fold, and responsive sizing based on screen width.

For accessibility, each image has appropriate alt text fetched from the database or generated from the service name. In carousel mode, screen readers announce the current image position and total count.

The AvailabilityToggle component

The AvailabilityToggle component renders the green availability toggle on Screen Three. Its purpose is to communicate to users whether you as the founder are currently accepting new client work for this specific service, building trust through transparent communication of capacity.

The component accepts an isAvailable boolean prop reflecting the service's current availability state, fetched from the is_available field in the services table. The component is read-only for end users, meaning tapping it has no effect.

Visually, the component renders as a horizontal layout containing a toggle visual on the left and a text label on the right. The toggle visual is a pill-shaped element approximately forty-eight pixels wide and twenty-six pixels tall, with rounded ends matching the pill shape. The toggle contains a small circle that sits on either the left or right side depending on state.

When isAvailable is true, the pill's background uses your lime highlight at full opacity. The circle inside is positioned on the right side of the pill, also coloured in your lime highlight but slightly lighter or with a subtle border for visual definition. The combined effect is a confidently green toggle in the on position.

When isAvailable is false, the pill's background uses your near-black at thirty percent opacity in light mode, and ceramic white at twenty percent opacity in dark mode. The circle inside is positioned on the left side, with a colour matching the page background. This creates a clearly off appearance.

To the right of the toggle visual with twelve pixels of spacing, the text label appears. When isAvailable is true, the label reads "Available" in regular Inter at fourteen pixels in your near-black colour. When isAvailable is false, the label reads "Currently unavailable" in the same styling, possibly with slightly muted opacity to reinforce the unavailable state.

The component is decorative rather than interactive for end users, but you as admin would have a separate admin interface for toggling this state in the database. When the database value updates, the component re-renders to show the new state.

For accessibility, the component has appropriate ARIA attributes despite being non-interactive. The component has role of status with aria-live polite, ensuring screen readers announce availability changes if they occur while the user is on the page.

The DetailFacts component

The DetailFacts component renders a grid of four facts about a service on Screen Three. Its purpose is to provide quickly scannable key attributes that help users understand the service's parameters at a glance, complementing the longer prose descriptions.

The component accepts an array of fact objects as its prop. Each fact object contains an icon

name corresponding to a Lucide icon, and a label string. Typically a service has exactly four facts, displayed in a two-by-two grid arrangement, but the component handles cases with fewer or more facts gracefully.

Visually, the grid uses CSS grid layout with two columns and as many rows as needed. On mobile, the two columns ensure facts remain readable without requiring the full screen width per fact. On desktop, the same two-column layout is maintained, possibly with adjustments to internal spacing.

Each fact within the grid is rendered as a horizontal row containing an icon on the left and a label on the right. The icon uses Lucide React, sized at sixteen pixels, in your deep violet colour at full opacity. The icon's container has a fixed width to ensure consistent alignment across all facts in the grid.

Beside the icon with eight pixels of spacing, the label appears in regular Inter at thirteen pixels with font weight of five hundred, in your near-black colour at eighty percent opacity in light mode. The label might wrap to a second line if it's long, with the wrapping aligned with the first line of text rather than the icon, creating clean indented text blocks.

The grid has internal spacing of sixteen pixels both horizontally between columns and vertically between rows, ensuring facts have room to breathe without becoming visually disconnected.

For accessibility, each fact pair is wrapped in a semantic element with appropriate role. The grid container has role of list and each fact has role of listitem, helping screen readers communicate the structured nature of the content.

The component does not require significant responsive adjustment because the two-column layout works well across screen sizes.

The LearnMoreTabs component

The LearnMoreTabs component renders the tab navigation at the top of the learn-more page that allows users to switch between Tab A literature content and Tab B deliverables list. Its purpose is to provide clear, simple navigation between two distinct content modes within the same page.

The component accepts an activeTab string prop indicating which tab is currently active, with values of "literature" or "deliverables". It also accepts a setActiveTab callback prop that the parent component provides for updating the active tab state when the user taps a different tab.

Visually, the component renders as a horizontal layout containing two tab labels separated by space. On mobile, the two tabs each take fifty percent of the available width, ensuring they're tappable across the full screen. On desktop, the tabs may be more compact and centred, depending on overall page design.

Each tab is rendered as a button with a label and a state-dependent visual treatment. The active tab's label appears in bold Inter at fifteen pixels with font weight of seven hundred, in your near-black colour at full opacity in light mode. Beneath the active tab's label sits a horizontal underline, two pixels thick, in your deep violet colour at full opacity. The underline spans the width of the label text plus a small amount of padding on each side.

The inactive tab's label appears in regular Inter at the same fifteen pixels but with font weight of four hundred and a colour of your near-black at fifty percent opacity. The inactive tab has no underline, creating clear visual differentiation from the active tab.

When the user taps an inactive tab, several things happen. The setActiveTab callback fires with the new tab’s identifier. The component re-renders to reflect the new active state, with the underline animating from one tab to the other if both tabs are visible. The underline transition takes two hundred milliseconds with an ease-out curve, providing smooth visual feedback.

For accessibility, the tabs use ARIA roles and attributes appropriate for tab interfaces. The container has role of tablist. Each tab button has role of tab with aria-selected reflecting whether it’s active. The content panels associated with each tab have role of tabpanel with aria-labelledby pointing to the corresponding tab. Keyboard navigation supports arrow keys to move between tabs and the Tab key to move from the tab list into the active panel’s content.

For responsive behaviour, the tabs maintain their horizontal arrangement on all screen sizes because the simple two-tab structure works well in any width.

The LearnMoreContent component

The LearnMoreContent component renders the actual content for whichever tab is currently active on the learn-more page. Its purpose is to display either the rich text literature or the structured deliverables list based on the active tab, handling the rendering of two quite different content types within a single component.

The component accepts a service object as its prop, containing all the data needed to render either tab’s content. It also accepts an activeTab string prop indicating which content to currently display.

When activeTab is “literature”, the component renders the long-form prose content from the service’s literature field. Because the literature is stored as TipTap JSON content, rendering it requires a TipTap rendering library or a custom renderer that handles the content’s structured format. The renderer converts the JSON into appropriate HTML elements with the correct styling.

The literature content is presented in a comfortable reading environment. The text container has a maximum width of approximately six hundred eighty pixels, even on larger screens, because long lines of text become tiring to read. On mobile, the container uses the full content width minus appropriate side padding. The text uses regular Inter at sixteen pixels with line height of approximately 1.7, providing comfortable reading rhythm. The colour is your near-black at full opacity in light mode and ceramic white in dark mode.

Within the literature, formatting is preserved from the rich text editor. Bold text uses font weight of seven hundred. Italic text uses italic style. Headings, if used, scale up appropriately, with H2 at twenty-two pixels weight 700 and H3 at eighteen pixels weight 600. Hyperlinks within the text appear in your deep violet colour with underline decoration, indicating they’re tappable. When a user taps a hyperlink within the literature, navigation occurs to the linked destination, which is typically another page within your application.

Lists within the literature, both ordered and unordered, render with proper indentation and spacing. Block quotes render with a subtle left border in your deep violet at fifty percent opacity and slightly muted text colour, distinguishing them from regular paragraphs.

When activeTab is “deliverables”, the component renders the structured list from the service_deliverables table. The list uses a softer visual style than typical bullet lists. Each item is

a row with an icon or marker on the left and the deliverable text on the right.
The marker for each deliverable is a small filled circle, ten pixels in diameter, in your lime highlight colour. This green marker creates a positive, confirmatory feel rather than the more clinical appearance of standard bullet points. To the right of the marker with twelve pixels of spacing, the deliverable text appears in regular Inter at fifteen pixels in your near-black at full opacity. The text wraps to multiple lines if needed, with the wrapping aligned with the first line rather than under the marker.
If a deliverable has an icon_name in the database, that icon replaces the simple lime circle. The icon uses Lucide React, sized at sixteen pixels, in your deep violet at full opacity. This option allows for more visually rich deliverable lists when desired, while the simple lime circle serves as the default.
Vertical spacing between deliverable items is sixteen pixels, providing comfortable separation that lets the eye move down the list without items feeling cramped. The list as a whole is wrapped in a container with maximum width similar to the literature content, maintaining consistent visual rhythm across both tabs.
For accessibility, the literature is rendered as semantic HTML with appropriate heading levels, paragraph elements, and list structures, allowing screen readers to navigate the content naturally. The deliverables list uses role of list with each item using role of listitem.
The component handles the transition between tabs through CSS opacity transitions, fading the previous content out and the new content in over one hundred fifty milliseconds. This brief transition prevents the jarring effect of instant content switching while remaining quick enough to feel responsive.
The PersistentBar component
The PersistentBar component renders the bottom bar that appears across Screens Two through Five, providing contextual navigation and the ever-present PAY button. Its purpose is to maintain commercial accessibility throughout the deeper screens of the user journey while adapting its content based on the user’s current location and the available content for the current service.
The component accepts a screen prop indicating which screen it’s currently rendered on, with values like “screen2”, “screen3”, “screen4”, or “screen5”. It reads from ServicesContext to access the current service and selected package. It uses these inputs together to determine what to render in its left and right slots.
Visually, the bar is a fixed element pinned to the bottom of the viewport, with full width matching the page’s content area. It has a height of approximately seventy-two pixels on mobile and eighty pixels on desktop, providing comfortable touch targets. The background uses your ceramic white at full opacity in light mode and a near-black variant in dark mode, with a subtle top border in your near-black at ten percent opacity providing visual separation from the page content above.
The bar has internal padding of sixteen pixels horizontally and twelve pixels vertically. Content is arranged in a horizontal layout with the left slot taking approximately forty percent of the bar’s width and the right slot taking sixty percent, although this distribution adapts based on content.
On Screen Two, where the bar is integrated into the slideup itself rather than rendered separately, the component might not be used. The slideup renders its own bottom area with the

gradient PAY button as previously specified. This is a design decision based on whether you want consistent component usage or context-appropriate variation.

On Screen Three, the bar’s left slot displays a contextual forward navigation link. The link’s content depends on whether learn-more content exists for the current service. If learn-more exists, the link shows text like “Read full process” with a small chevron-right icon to its right. The text uses Inter at thirteen pixels with font weight of six hundred, in your deep violet colour. The icon is sized at fourteen pixels in matching colour. Tapping this link navigates to the learn-more page through Next.js routing.

If no learn-more content exists for the service, the left slot’s contextual link is replaced with a small icon button. This button uses an info icon or ellipsis icon from Lucide, sized at twenty pixels, in your near-black at sixty percent opacity. The button has a tap target of approximately forty pixels square. When tapped, a small popup menu appears containing links to Terms of Service and Privacy Policy. The menu has a white background with subtle shadow, rounded corners, and links styled as standard text links in your deep violet.

The right slot of the bar displays the solid PAY button. The button has a width fitting its content with appropriate padding, approximately twenty pixels horizontal padding internally. Height matches the bar’s effective content height. The background uses your deep violet colour at full opacity. The text reads “PAY” followed by the formatted price, in uppercase Inter at fifteen pixels with font weight of seven hundred, in your ceramic white colour. The button has rounded corners with an eight-pixel radius, a slightly more compact rounding than the gradient button on the slideup, giving it a cleaner business-like feel.

The price displayed updates in real-time based on the selected package in ServicesContext. If the user changes their package selection through deep linking or by navigating back to the slideup, the price here updates accordingly.

Tapping the PAY button opens the ContactPopup. The popup overlays the current screen without changing the URL, so the persistent bar continues to be visible behind the popup if the popup doesn’t fully cover it.

On Screens Four and Five, the bar’s left slot displays a back-context navigation link. The link reads “Back to overview” with a small chevron-left icon to its left, using the same styling as the forward link on Screen Three but pointing in the opposite direction. Tapping this link navigates back to the Service Detail page using Next.js routing. This design ensures that users at the deepest level can quickly return to the service overview, treating the back navigation as a feature rather than relying solely on the browser’s back button.

The right slot continues to show the PAY button with the same styling and behaviour as on Screen Three.

For accessibility, the bar has appropriate ARIA attributes including role of banner if treated as primary navigation, or role of region with appropriate label if treated as supplementary. Each interactive element within the bar has appropriate aria-label and is keyboard focusable.

For responsive behaviour, the bar adapts its dimensions, padding, and font sizes based on screen size. On very narrow screens, the contextual text might be shortened or replaced with icons to ensure both slots remain visible without overflow. The PAY button always remains visible

and maintains its prominent appearance.

The ContactPopup component

The ContactPopup component renders the WhatsApp handoff popup that appears when a user taps any PAY button. Its purpose is to provide a warm, human transition from the application into a real conversation with you on WhatsApp, while collecting just enough context to make that conversation immediately productive.

The component accepts several props. The service object prop provides the service the user is enquiring about. The packageId prop indicates which specific package they had selected. The isOpen boolean prop controls visibility, and the onClose callback prop fires when the user dismisses the popup.

Visually, the popup presents as a centred modal with substantial visual weight. On mobile, the popup takes approximately ninety percent of the screen width with appropriate vertical sizing based on content. On desktop, the popup has a maximum width of approximately five hundred pixels, centred horizontally on the screen. The popup is positioned vertically centred on the viewport, although it should adjust upward slightly to account for keyboard appearance on mobile when input fields are focused.

The popup has a ceramic white background in light mode and a near-black variant in dark mode. It has rounded corners with a twenty-four-pixel border radius, creating a soft friendly shape. It casts a moderate drop shadow to lift it above the page content, with the shadow using black at fifteen percent opacity, blur of twenty-four pixels, and offset of zero horizontal and four pixels vertical.

The popup has internal padding of twenty-four pixels on mobile and thirty-two pixels on desktop, ensuring content has comfortable breathing room. A backdrop overlay appears behind the popup, dimming the page content. The backdrop uses black at fifty percent opacity.

When the popup opens, the animation takes two hundred fifty milliseconds with an ease-out curve. The popup scales up from ninety-five percent to one hundred percent while fading in from zero opacity to full. Simultaneously the backdrop fades in. The combined animation feels gentle rather than aggressive, matching the warm tone the popup is designed to communicate.

Inside the popup, content is arranged from top to bottom in a deliberate sequence. At the very top, the trust block establishes the human nature of the interaction. This block contains a circular profile photo of you and accompanying text identifying you as a real person.

The profile photo is a circular image with diameter of approximately seventy-two pixels. It’s positioned with appropriate top spacing and centred horizontally or aligned to the left depending on overall popup layout. Around the photo sits a thin gradient ring, two pixels thick, using your gradient brand colours from lavender to deep violet. The ring ties the trust block to your overall brand identity and signals visual quality. The photo itself comes from a static asset in your codebase, optimised and properly sized.

Beside or below the photo, your name and a brief identifier appear. The name uses bold Inter at sixteen pixels with font weight of seven hundred, in your near-black at full opacity. Below the name with four pixels of spacing, a smaller identifier appears like “Founder, CKA Visuals” in regular Inter at thirteen pixels in your near-black at sixty percent opacity. This combination

establishes credibility and humanity in just a few elements.
Below the trust block with twenty-four pixels of vertical spacing, the explanatory message appears. The message is written in conversational, warm tone, explaining honestly why the WhatsApp handoff exists. The text might read something like: “Hey, before you pay, let’s chat first. We’re still working on getting smooth payment processing on the website, so for now I personally handle every project enquiry through WhatsApp. It’s actually how I prefer it because it lets me understand exactly what you need before any money changes hands.”
The text uses regular Inter at fifteen pixels with comfortable line height of 1.6, in your near-black at full opacity for the primary text. The colour is slightly muted, perhaps at ninety percent opacity, to feel softer than headline text. The text wraps naturally within the popup width.
Below the explanation with twenty pixels of spacing, the input fields appear. There are two input fields, ordered with project description first and name second, contrary to typical form ordering, because the warm message has already established context and the user’s mind is on their project rather than introducing themselves.
The project description field is a multi-line textarea with approximately three or four rows of visible height. It has a placeholder text reading “What kind of project do you have in mind?” in your near-black at forty percent opacity. The field has internal padding of twelve pixels, a one-pixel border in your near-black at fifteen percent opacity, and rounded corners with an eight-pixel radius. The font is regular Inter at fifteen pixels in your near-black at full opacity.
When the field is focused, the border colour transitions to your deep violet at full opacity over one hundred fifty milliseconds, providing visual feedback that the field is ready for input. The field’s background remains the same.
Below the project description with twelve pixels of spacing, the name field appears. This is a single-line input with placeholder text reading “What should I call you?”. Its styling mirrors the project description field including border, padding, and focus behaviour. The casual placeholder phrasing reinforces the warm, personal tone of the popup.
Below the name field with twenty-four pixels of spacing, the primary action button appears. This button is full-width within the popup’s content area, with a height of approximately fifty pixels and rounded corners using a twelve-pixel radius. The background uses the recognisable WhatsApp green colour, specifically the hex value 25D366 which is WhatsApp’s official brand colour. This deliberate use of a non-CKA brand colour is strategic because users have strong existing associations with this specific green and what it represents.
The button’s content includes an icon and text. The WhatsApp icon, either from a Lucide component or a custom SVG, sized at twenty pixels in white. To the right of the icon with eight pixels of spacing, the button text reads “Continue on WhatsApp” in bold Inter at fifteen pixels with font weight of seven hundred, in white at full opacity.
When the user taps this button, several actions happen in sequence and they happen quickly so they feel simultaneous. First, validation checks ensure both input fields contain content. If either is empty, that field highlights with a subtle red border using your hot pink at full opacity, and the button doesn’t proceed. If both are filled, the actions continue.
Second, an enquiry row is written to the enquiries table in the database through the

useEnquirySubmit hook. The row includes the service id, package id, the user's name, the project description, and a timestamp. This write is asynchronous but doesn't block the next steps, ensuring the user experience remains responsive even if the database write is slow.
Third, the WhatsApp click-to-chat URL is constructed using the whatsapp utility. The URL is in the format "https://wa.me/237671122318?text=" followed by URL-encoded message content. The message content is constructed from a template that includes the user's name, the service name, the package name, the price, and the project description. An example of the constructed message might read: "Hi, I'm Sarah and I'm interested in Logo Concepts - New Logo for 30,000 XAF. Here's what I'm working on: I'm starting a new café in Buea and need a fresh, modern logo that captures the local character."
Fourth, the WhatsApp URL is opened in a new browser tab using window.open, allowing the user to return to your application after sending the message. On mobile, this typically opens the WhatsApp app directly through the deep link. On desktop, this opens WhatsApp Web in a new tab.
Below the primary action button with sixteen pixels of spacing, a secondary "Maybe later" text link appears. This link is a simple text element in regular Inter at thirteen pixels in your near-black at sixty percent opacity. Tapping this link triggers the onClose callback, dismissing the popup without taking any action.
For accessibility, the popup uses appropriate ARIA attributes. Role is dialog, aria-modal is true, and aria-labelledby points to the title element if there is one. Keyboard focus is trapped within the popup while it's open, with focus initially placed on the first input field. The Escape key dismisses the popup.
For responsive behaviour, the popup adapts its dimensions and internal spacing based on screen size. On mobile, the popup may take more of the screen width and adjust vertically based on keyboard appearance. On desktop, the popup maintains its centred modal appearance with consistent maximum width.
Part Six — WhatsApp Integration in Detail
The WhatsApp integration is a critical and somewhat unusual piece of business logic that deserves detailed explanation. This section explains how the integration works from technical implementation through user experience.
The integration relies on WhatsApp's click-to-chat feature, which is a URL format that opens a WhatsApp conversation with a specific phone number and pre-fills a message. The URL format is "https://wa.me/" followed by the phone number in international format without the plus sign or any spaces, followed by "?text=" and the URL-encoded message content. For your number plus 237 671 122 318, the URL begins with "https://wa.me/237671122318".
The message content needs to be URL-encoded to handle special characters and spaces. Encoding converts characters like spaces to %20, line breaks to %0A, and various punctuation marks to their encoded equivalents. JavaScript provides a built-in encodeURIComponent function that handles this encoding automatically.
The message template that gets pre-filled into WhatsApp is generated dynamically based on the user's inputs and the service context. The template structure follows a pattern designed to feel

like the user wrote it themselves rather than appearing as obviously automated. The template begins with a greeting using the user's name, then states their interest in the specific service and package with price, then includes their project description.

A complete example of a generated message before encoding might read: "Hi, I'm Marcus and I'm interested in Logo Concepts - Redesign for 20,000 XAF. Here's what I'm working on: I run a small electronics store in Lagos that's been operating for five years and the current logo feels dated. Looking for something that feels modern but still trustworthy."

After encoding, this message becomes a long URL string with all spaces and special characters converted to their encoded forms. When the user's browser opens this URL, WhatsApp recognises the click-to-chat format and opens with the message ready to send.

The encoded message has a length limit imposed by WhatsApp's URL handling, typically around two thousand characters for the encoded form, which translates to roughly five hundred characters of natural text. Your message template should stay well within this limit even with longer project descriptions, but the implementation should include a safety check that truncates excessive descriptions with an ellipsis to ensure URLs always remain functional.

The user's experience when tapping the "Continue on WhatsApp" button differs slightly between mobile and desktop. On mobile, if the user has WhatsApp installed, the deep link opens the WhatsApp app directly with the conversation prepared. If WhatsApp is not installed, the user is taken to a web page with options to install WhatsApp or continue in WhatsApp Web. On desktop, the link opens WhatsApp Web in a new browser tab, which requires the user to be logged into WhatsApp Web through a previous QR code scan, or they're prompted to scan the QR code first.

The enquiry logging happens regardless of whether the user actually proceeds to send the WhatsApp message. The button tap triggers both the database write and the WhatsApp URL opening. If the user taps the button, then changes their mind and never sends the message, the database still has a record of their intent. This data is valuable for understanding conversion patterns and following up if appropriate.

The phone number plus 237 671 122 318 should be stored as a constant in your codebase rather than hardcoded into multiple places, making it easy to update if your contact number ever changes. The constant lives in the whatsapp.js library file along with the message template logic.

Part Seven — Implementation Order

The order in which the developer builds the section significantly affects the success of the implementation. Building in the wrong order leads to wasted work, integration problems, and frustration. Here is the recommended sequence with reasoning for why each step belongs where it does.

The implementation begins with database setup. The developer creates all six tables plus the join table, applies the row-level security policies, and creates the two storage buckets with their access policies. Until the database is in place, no UI work can be tested with real data. Building UI against placeholder data leads to surprises when real data is finally connected because real data has edge cases and patterns that placeholders don't capture.

After the database structure exists, the developer populates it with seed data. This means at least one category, two services within that category, two packages per service, two tags

assigned to those services, four deliverables per service, and uploaded images for the illustrations and hero images. The seed data should represent realistic content rather than placeholder text like “Lorem ipsum”, because realistic content reveals layout issues and typography problems that placeholder text hides.

With the database populated, the developer builds the data-fetching hooks. These hooks contain pure logic with no UI, making them easier to test and debug in isolation. The four primary hooks are useServiceCategories, useCategoryServices, useService, and useEnquirySubmit. Each hook should handle loading states, error states, and success states explicitly, providing clear feedback to whatever UI consumes them.

Next comes the ServicesContext provider. The context establishes the state management foundation that subsequent components will read from. Building the context before the components that depend on it ensures the context’s API is well-thought through, rather than being designed reactively as components encounter problems.

With infrastructure in place, the developer begins building UI from the most discoverable surface inward. Screen One comes first because it’s the entry point, and getting it right validates the data flow from database through hooks through context into actual rendered components. The order within Screen One is the CategoryBand component first since it determines what other content displays, then the ServiceTags component, then the ServiceCard component, and finally the page layout that composes them together.

After Screen One works correctly with real data, the slideup follows. The order within the slideup is the ServiceSlideup container component first to establish the overlay behaviour, then the PackageBox component, then the PackageList that composes multiple package boxes, and finally the integration of all elements within the slideup including the hero carousel through ServiceHero.

The ContactPopup is built next because it’s the destination of the PAY action that exists on the slideup. Building the popup at this point allows end-to-end testing of the WhatsApp handoff before more screens are built. This early integration catches any issues with the WhatsApp URL format, message construction, or database logging while the codebase is still simple.

Screen Three follows, including the dynamic route at /services/[id], the data fetching for a single service, the static variant of ServiceHero, the AvailabilityToggle, the DetailFacts, and the PersistentBar in its Screen Three configuration. The PersistentBar’s contextual navigation logic should be built carefully here, with clear handling of cases where learn-more content exists versus doesn’t.

The learn-more page comes next, including the dynamic route at /services/[id]/learn-more, the LearnMoreTabs component, the LearnMoreContent component handling both literature and deliverables rendering, and the PersistentBar in its Screens Four and Five configuration with back-context navigation.

After all screens are built and connected, the developer focuses on transitions and animations. This includes refining the slideup’s slide animation, ensuring smooth transitions between screens, polishing the tab switching animation, and fine-tuning any other motion design throughout the section.

Finally, the developer conducts comprehensive testing across the complete flow. This includes testing on mobile devices and desktop, in light and dark modes, with both authenticated admin user and unauthenticated visitor, with services that have learn-more content and services that don't, and with various edge cases like services with one package versus multiple packages, services with no deliverables, services that are unavailable, and so on. Each scenario should be walked through deliberately, with any issues fixed before declaring the implementation complete.

TAKE NOTE OF THIS SECTION BELOW:

Admin Interface Specification

The admin interface for the Services section follows the same hidden-access pattern already established in your application. You long press the avatar, log in, and edit mode activates. On the Services pages specifically, edit mode reveals additional controls that allow you to manage every piece of content without touching the database directly.

The philosophy governing the admin interface is inline editing wherever possible. Rather than a separate admin dashboard at a different URL, controls appear directly on the pages themselves when you're logged in. This means what you see as admin is exactly what your users see, with editing affordances layered on top. You're never designing in the dark, wondering what something will look like when published.

The Services Discovery Page in Admin Mode

When you arrive at the Services discovery page as admin, two additional elements appear that regular users never see. A floating action button appears at the bottom left of the screen, using your deep violet background and a plus icon in ceramic white. This button opens a sheet for creating a new service category. Below each category band, a small edit icon appears in your deep violet colour. Tapping it opens an inline editor for that category's name and tagline.

Each service card in admin mode gains a thin deep violet border to indicate it's editable. A small toolbar appears at the top right corner of each card containing three icon buttons. The first is a pencil icon that opens the service editor sheet. The second is an eye icon that toggles the service's is_visible field, with the icon appearing crossed out when the service is hidden. The third is a drag handle icon, three horizontal lines, that enables reordering services within the category through press and drag.

The service editor sheet slides up from the bottom and contains all the fields needed to manage a service card. The sheet has your standard dark header with the title "Edit Service" and a close button. Below the header, fields appear in a logical sequence that mirrors the visual hierarchy of the card itself.

The first field is the service name, a single line text input with the label "Service Name" in your muted label style above it. The second field is the short description, a three-line textarea for the card's brief text. The third field is the long description, a five-line textarea for the slideup and detail page text. The fourth field is the starting price, a numeric input with a currency selector beside it defaulting to XAF. The fifth element is the illustration picker, showing the current

illustration as a thumbnail with a “Change Illustration” button beneath it that opens the device’s file picker.

Below these basic fields, the sheet continues into a section for managing packages. The section has a label reading “Deliverable Variants” and shows existing packages as compact rows, each displaying the variant name, price, and a delete button. At the bottom of the package list sits an “Add Variant” button that expands an inline form for adding a new package with fields for name, price, currency, delivery time, and the featured toggle.

Below packages sits a toggle for service availability with the label “Currently Accepting Projects” and the green toggle visual you’ve already designed. Below that sits a toggle for service visibility with the label “Visible to Users”.

At the bottom of the sheet sit two buttons side by side. The left button is a plain text “Cancel” link in muted styling. The right button is a solid deep violet “Save Changes” button. Tapping save writes all the changes to the relevant database tables simultaneously and closes the sheet.

The Service Detail Page in Admin Mode

On Screen Three, admin mode reveals an edit bar that appears below the hero image. The bar has a subtle deep violet background at fifteen percent opacity and contains a pencil icon on the left with the text “Edit Page Content” beside it. Tapping this bar opens the detail content editor.

The detail content editor is a full-screen sheet because the content it manages is substantial. The sheet header reads “Service Detail Content” with your standard close button. Inside, the sheet is divided into clearly labelled sections.

The first section is “Hero Images” and shows the current hero images as a horizontal row of thumbnails. Each thumbnail has a small delete button in its top right corner. At the end of the row sits an “Add Image” button that opens the file picker. Images can be reordered by pressing and dragging their thumbnails.

The second section is “Availability” containing the same toggle as on the service editor, controlling the is_available field.

The third section is “Bundle Name” containing a single text input. When filled, the “Included in [bundle name]” badge appears on the page. When empty, the badge doesn’t appear.

The fourth section is “Detail Facts” showing the four facts as editable rows. Each row has an icon picker on the left, a text input for the label, and a delete button. An “Add Fact” button at the bottom adds new rows up to the maximum of four.

The save behaviour matches the service editor, with changes written on confirmation.

The Learn More Page in Admin Mode

On the learn-more page, admin mode reveals edit buttons at the top of each tab’s content area. On Tab A, a button reading “Edit Literature” opens TipTap, your rich text editor, in a full-screen sheet. TipTap provides formatting controls in a toolbar at the top including bold, italic, headings, links, and lists. The editing surface below is a clean white area where you write and format directly. The save button at the top right of the sheet writes the TipTap JSON to the literature field in the database.

The link insertion in TipTap allows you to highlight text and create a hyperlink. When inserting a link, a small popup appears with a URL field. For links within your own application like a link to

Screen Four from Screen Three's content, you type the relative path like "/services/[id]/learn-more". The editor handles these internal links correctly in the rendered output.
On Tab B, a button reading "Edit Deliverables" opens a sheet showing the deliverables list as editable rows with the same icon picker and label input pattern from the detail facts editor. An "Add Deliverable" button allows new items. Rows are reorderable through drag. Save writes all changes to the service_deliverables table.

WhatsApp Popup Copy
Here is the complete copy for the ContactPopup, written in the warm business-casual voice you described.
The heading that appears above your photo reads nothing, because the visual trust block speaks for itself. Your name below the photo reads "Chidera Kang" or whatever your preferred professional name is, and beneath it sits "Founder · CKA Visuals".
The body message reads as follows, and this is the full text that appears in the popup:
Body message:
"Hey 👋 tapping that button is going to open a WhatsApp chat with us directly.
We're still sorting out payments on the website, so for now we handle every project personally through WhatsApp. It's actually a better experience. You speak with a real person, we agree on everything together, and then we move from there.
Just fill in below and we'll take it from there."
Trust strip:
"You're speaking directly with the CKA Visuals team. No bots, no middlemen."
Project field placeholder:
"Tell us briefly what you're working on."
Name field placeholder:
"What's your name?"
Button:
"Continue on WhatsApp →"
Cancel link:
"Maybe later"

Autogenerated WhatsApp message:
"Hi, my name is [Name] 👋
I'm reaching out about your [Service Name] service — specifically the [Package Name] at [Price].
Here's a bit about what I need:
[Project Description]
Looking forward to hearing from you."